

ReactJS

React Hook Form:

What is React Hook Form?

React Hook Form is a library for managing forms in React applications using React Hooks. It helps handle:

- Form state management
- Input registration
- Validation
- Error handling
- Form submission

while minimizing unnecessary re-renders, making forms faster and more performant than many traditional approaches.

Why Use React Hook Form?

1. Less code
2. Built-in Validation
3. Better Performance
4. Works well with Large Forms

How to install it?

Command:

```
npm install react-hook-form
```

Now, let's see an example using the code...!

App.jsx: a basic form.

```
src > App.jsx > App
1 import React from 'react'
2 import { useForm } from 'react-hook-form'
3 import { data } from 'react-router-dom';
4
5 function App() {
6   const {
7     register,
8     handleSubmit,
9     watch,
10    formState: {errors},
11  } = useForm();
12  function onSubmit(data){
13    console.log("Submitting the form",data);
14  }
15
16  return (
17    <form onSubmit={handleSubmit(onSubmit)}>
18      <div>
19        <label>First Name:</label>
20        <input {...register('firstName')}/>
21      </div>
22      <br/>
23      <div>
24        <label>Middle Name:</label>
25        <input {...register('middleName')}/>
26      </div>
27      <br/>
28      <div>
29        <label>Last Name:</label>
30        <input {...register('lastName')}/>
31      </div>
32      <input type='submit' />
33    </form>
34  )
35 }
36 export default App
```

Call useForm() and give me the tools (register, handleSubmit, watch, and errors) that I need to manage my form

register() connects an input field to React Hook Form so it can track, store, and validate that field's value. It essentially tells React Hook Form, "Monitor this input and include its value in the form data when the form is submitted."

This code creates a simple form in React using React Hook Form that collects a user's first name, middle name, and last name. The useForm() hook is used to manage the form, and the register() function connects each input field to React Hook Form so it can track the values entered by the user without needing separate useState variables. When the user clicks the submit button, handleSubmit(onSubmit) gathers all the input values into a single object and passes it to the onSubmit function, which then logs the collected data to the console. For example, if the user enters "John", "Michael", and "Doe", the console will show { firstName: "John", middleName: "Michael", lastName: "Doe" }. In short, this code demonstrates how React Hook Form simplifies form handling by automatically managing form data collection and submission with minimal code.

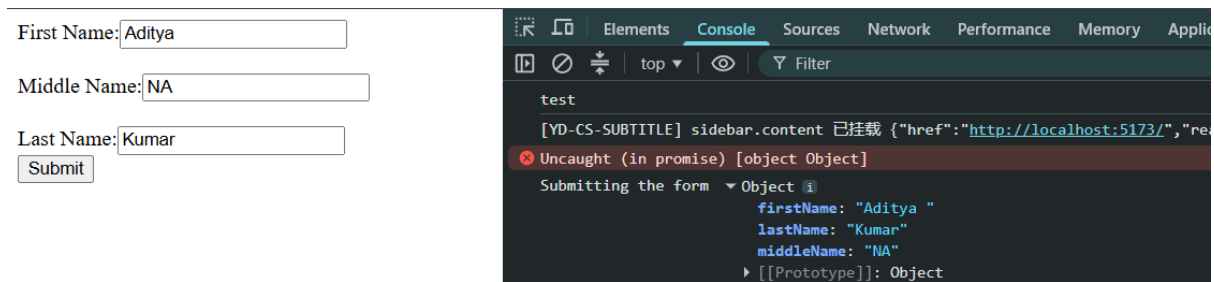
Output:

First Name:

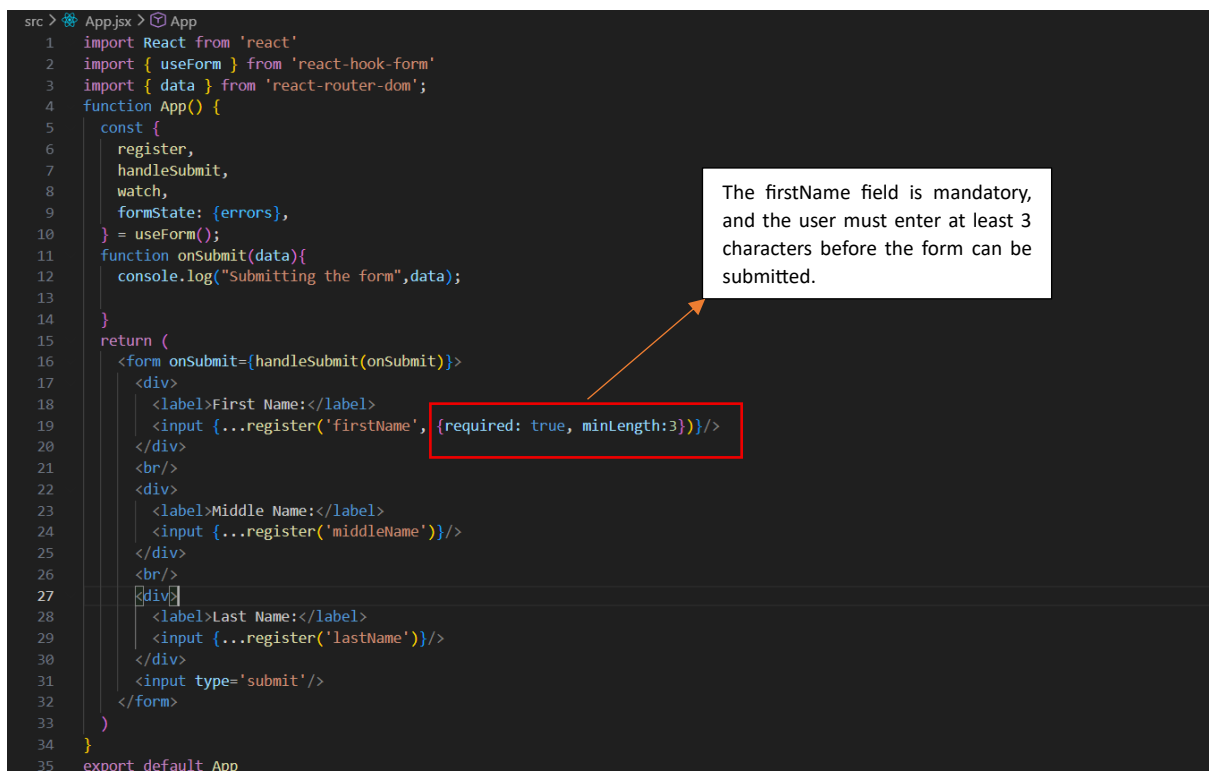
Middle Name:

Last Name:

Output: when filled and clicked on the submit button.



Now, let's talk about the validation, and restriction. Suppose I want that the first name shall be mandatory and its minimum length be 3.



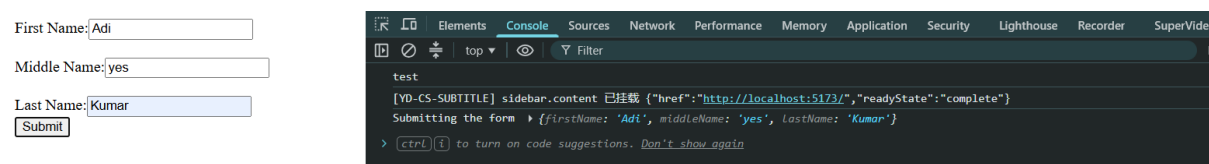
Output:

First Name:

Middle Name:

Last Name:

Output: it will allow the submit only when this condition is met (condition of required field, minimum length)



Now, we can also add some error message for the case the validation is not met. How?

App.jsx:

```
src > App.jsx > App
1  import React from 'react'
2  import { useForm } from 'react-hook-form'
3  import { data } from 'react-router-dom';
4  function App() {
5    const {
6      register,
7      handleSubmit,
8      watch,
9      formState: { errors },
10     } = useForm();
11     function onSubmit(data){
12       console.log("Submitting the form",data);
13     }
14     return (
15       <form onSubmit={handleSubmit(onSubmit)}>
16         <div>
17           <label>First Name:</label>
18           <input {...register('firstName',
19             {
20               required: true,
21               minLength: (value:3, message:"Min Len atleast 3")
22             })}/>
23           {errors.firstName && <p>{errors.firstName.message}</p>}
24         </div>
25         <br/>
26         <div>
27           <label>Middle Name:</label>
28           <input {...register('middleName')}/>
29         </div>
30         <br/>
31         <div>
32           <label>Last Name:</label>
33           <input {...register('lastName')}/>
34         </div>
35         <input type='submit' />
36       </form>
37     )
38   }
39 }
```

Defines validation rules for the firstName field: the field is required and must contain at least 3 characters, otherwise the custom error message is shown.

Checks if the firstName field has a validation error, and if it does, displays the corresponding error message on the screen.

Output: when I try to submit with two characters only.

First Name:

Min Len atleast 3

Middle Name:

Last Name:

Rules for ReactJS Hook:

The Rules of Hooks in React ensure that hooks work consistently and correctly.

1. Only Call Hooks at the Top Level
2. Only Call Hooks from React Functions
3. Custom Hook Names Must Start with use
4. Follow the Dependency Rules for Effects

Note: A custom hook can use React's built-in hooks (useState, useEffect, etc.), and a custom hook can also call another custom hook.

Custom Hooks:

Custom Hooks are reusable JavaScript functions that let you share stateful logic between React components. Their names must start with use.

Benefits: Reusability, cleaner components, and separation of logic from UI.

A custom hook:

- Starts with the word use
- Can use other React hooks (useState, useEffect, useContext, etc.)
- Can call other custom hooks
- Returns values, functions, or objects that components can use

Syntax:

```
function useCustomHook() {  
  // Hook logic  
  return value;  
}
```

Component	Custom Hook
Returns JSX	Returns data/functions
Handles UI	Handles logic
Rendered by React	Called inside components/hooks

Also,

Prebuilt (Built-in) Hooks	Custom Hooks
Provided by React.	Created by developers.
Imported directly from React.	Written using built-in hooks and/or other custom hooks.
Examples: useState, useEffect, useContext, useReducer.	Examples: useCounter, useFetch, useLocalStorage.
Used to manage state, effects, context, etc.	Used to encapsulate and reuse application-specific logic.
Part of React API.	Not part of React; user-defined.
Cannot be modified.	Can be customized as needed.

Example: a basic custom hook, useToggle, to hide and show the text.

App.jsx:

```
src > App.jsx > App
1  import React from 'react'
2  import useToggle from './myComponent/useToggle'
3
4  function App() {
5    const [value,toggleValue] = useToggle(true);
6    console.log("Val:", value);
7    return (
8      <div>
9        <button onClick={toggleValue}>Toggle heading</button>
10       {
11         value? <h1>Aditya</h1>:null
12       }
13     </div>
14   )
15 }
16
17 export default App
```

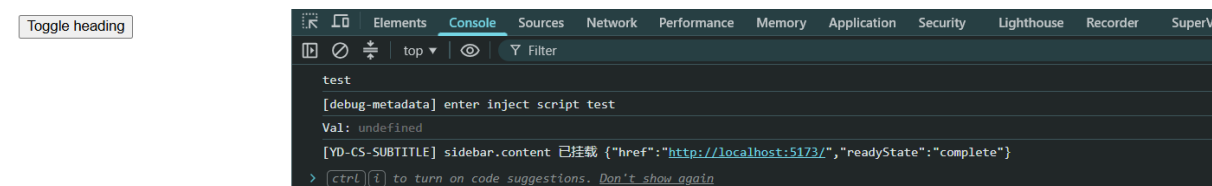
This component uses the useToggle custom hook to manage the visibility of a heading. The hook returns a state variable value and a function toggleValue. When the "Toggle heading" button is clicked, toggleValue is called, which changes the state between true and false. The heading "Aditya" is conditionally rendered only when value is true; otherwise, nothing is displayed. The current state value is also logged to the console on each render.

useToggle.jsx:

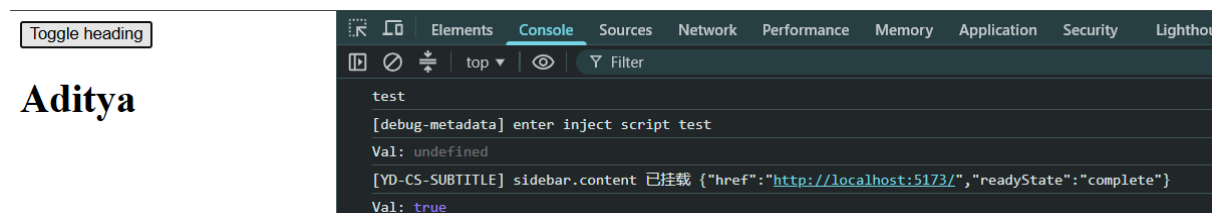
```
src > myComponent > useToggle.jsx > default
1  import { useState } from "react";
2
3  const useToggle=(defaultVal)=>{
4    const [value,setValue] = useState();
5    function toggleValue(val){
6      if(typeof val !== 'boolean'){
7        setValue(!value);
8      } else{
9        setValue(val);
10      }
11    }
12    return [value,toggleValue];
13  }
14
15  export default useToggle;
```

The useToggle custom hook manages a boolean-like state and returns the current value along with a function to update it. It uses React's useState hook to store value, and the toggleValue function either toggles the current state when no boolean is provided, or sets the state directly when a boolean is passed. The hook returns [value, toggleValue], allowing components to easily show, hide, or toggle UI elements. (Note: defaultVal is currently unused; it should be passed to useState(defaultVal) if you want an initial value.)

Output: on load.



Output:



Output:

Toggle heading

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurityLighthouse

topFilter

test

[debug-metadata] enter inject script test

Val: undefined

[YD-CS-SUBTITLE] sidebar.content 已挂载 {"href":"http://localhost:5173/","readyState":"complete"}

Val: true

Val: false

API:

What is API?

In React, an API (Application Programming Interface) is usually a way for your React application to communicate with a server or another application to get or send data.

Common API Methods

- **GET** – Retrieve data.
- **POST** – Create new data.
- **PUT/PATCH** – Update existing data.
- **DELETE** – Remove data.

Popular Ways to Call APIs in React

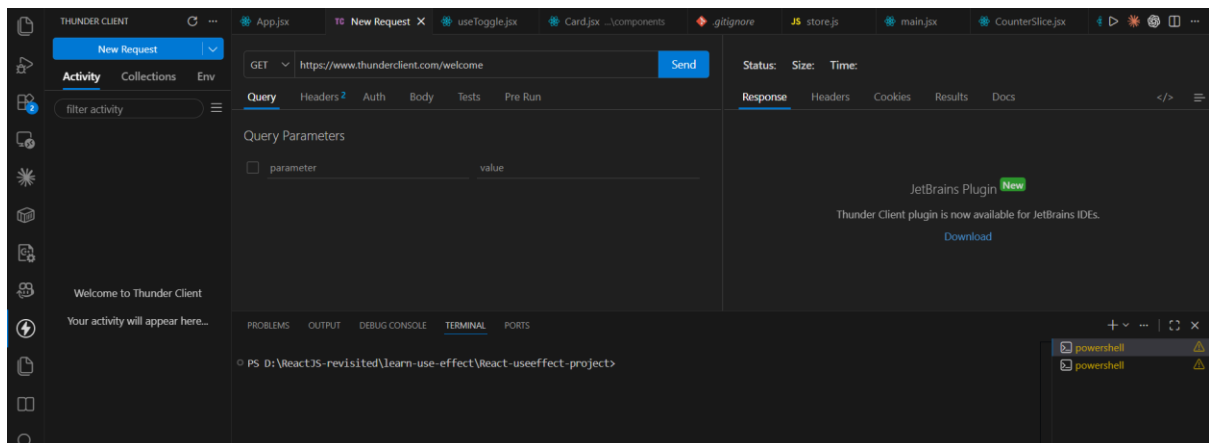
- Built-in `fetch()`
- [Axios](#) (popular HTTP client library)

Simple Rule:

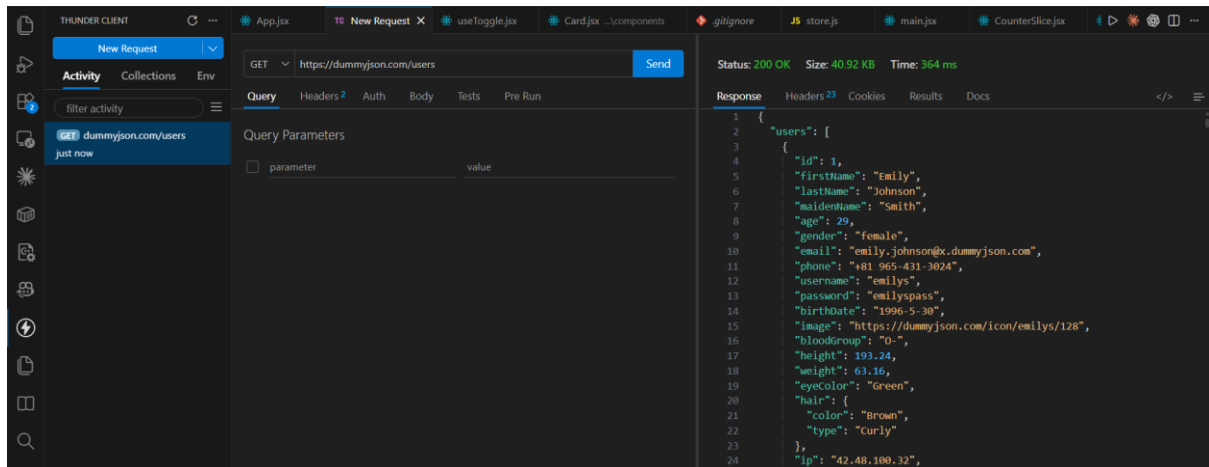
- Browser JavaScript (React) usually needs APIs because it cannot safely access databases directly.
- Node.js can access databases directly, so it may not need another API.
- Both JavaScript and Node.js can call APIs when they need data from another service.

So, suppose if we have an API URL to work on, then first we will test if the API is working properly or not.

We can use POSTMAN, etc tools. But we are here using the Thunder Client extension provided in the VS code. The interface which opens:

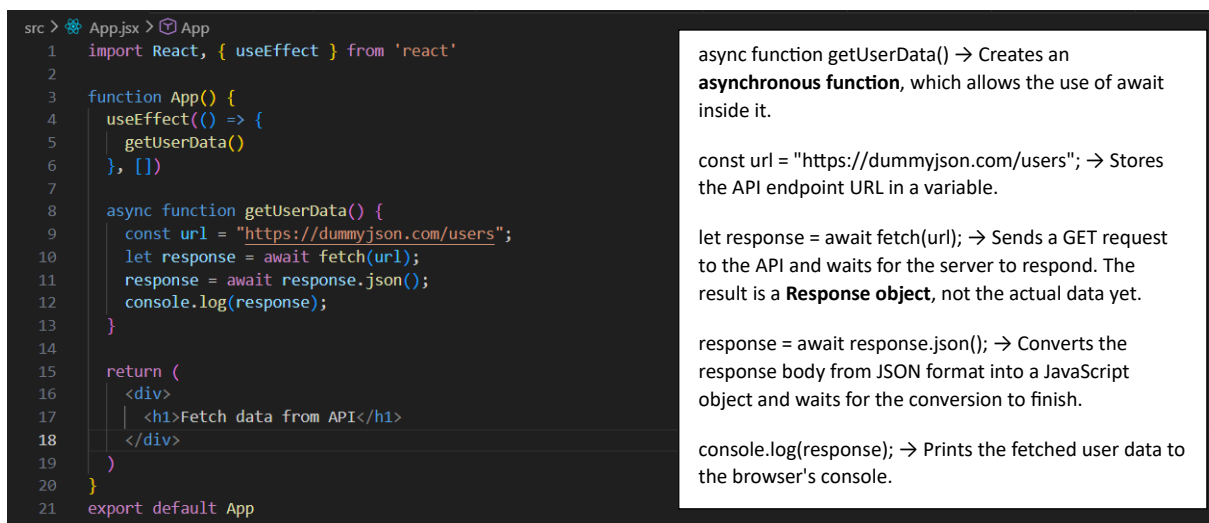


Now, just use the dummy API: <https://dummyjson.com/users> . Simply paste it, and “GET” and then send the request.



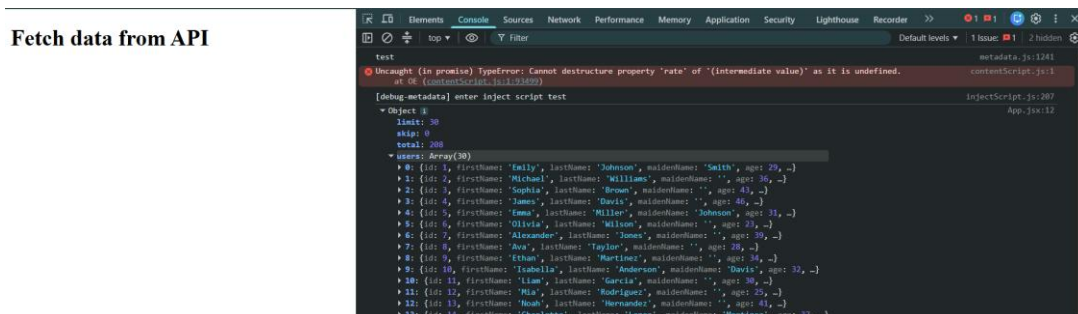
Now, we will see how to use it in code.

App.jsx:



This React component uses `useEffect()` with an empty dependency array (`[]`) so that `getUserData()` runs only once when the component first loads. The `getUserData()` function is asynchronous; it uses `fetch()` to send a GET request to the DummyJSON users API, waits for the response, converts the returned JSON data into a JavaScript object using `response.json()`, and then prints the user data to the browser console.

Output:



Also, you can see that there are coming some extra information such as the limit, skip, etc. in order to get only the users data. We can use `useState` and then use the `response.users`.

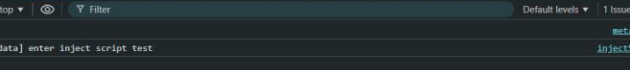
App.jsx:

```
src > App.jsx > App
1 import React, { useEffect, useState } from 'react'
2
3 function App() {
4   const [usersData, setUsersData] = useState([]);
5
6   useEffect(() => {
7     getUserData();
8   }, [])
9
10  async function getUserData() {
11    const url = "https://dummyjson.com/users";
12    let response = await fetch(url);
13    response = await response.json();
14    setUsersData(response.users);
15  }
16
17  console.log(usersData);
18
19  return (
20    <div>
21      <h1>Fetch data from API</h1>
22    </div>
23  )
24 }
25 export default App
```

This React component uses `useState([])` to create a state variable called `usersData`, which initially stores an empty array. When the component loads for the first time, `useEffect()` calls the `getUserData()` function, which fetches user data from the API, converts the response from JSON into a JavaScript object, and then stores the `users` array from the response into the `usersData` state using `setUsersData()`. Updating the state causes React to re-render the component with the new data. The `console.log(usersData)` statement prints the current state to the console, so you'll first see an empty array and then the fetched users after the API request completes.

Output:

Fetch data from API



```
test
[debug-metadata] enter inject script test
>

metadata.js:1241
injectScript.js:287
App.js:17
App.js:17
top.js:17

[{"id": 1, "firstName": "Emily", "lastName": "Johnson", "maidenName": "Smith", "age": 29, ...},
{"id": 2, "firstName": "Michael", "lastName": "Williams", "maidenName": "", "age": 36, ...},
{"id": 3, "firstName": "Sophia", "lastName": "Brown", "maidenName": "", "age": 43, ...},
{"id": 4, "firstName": "James", "lastName": "Davis", "maidenName": "", "age": 46, ...},
{"id": 5, "firstName": "Anna", "lastName": "Miller", "maidenName": "Johnson", "age": 31, ...},
{"id": 6, "firstName": "Olivia", "lastName": "Wilson", "maidenName": "", "age": 23, ...},
{"id": 7, "firstName": "Alexander", "lastName": "Jones", "maidenName": "", "age": 39, ...},
{"id": 8, "firstName": "Ava", "lastName": "Taylor", "maidenName": "", "age": 28, ...},
{"id": 9, "firstName": "Ethan", "lastName": "Martinez", "maidenName": "", "age": 34, ...},
{"id": 10, "firstName": "Isabella", "lastName": "Anderson", "maidenName": "Davis", "age": 32, ...},
{"id": 11, "firstName": "Liam", "lastName": "Garcia", "maidenName": "", "age": 30, ...},
{"id": 12, "firstName": "Mia", "lastName": "Rodriguez", "maidenName": "", "age": 25, ...},
{"id": 13, "firstName": "Noah", "lastName": "Hernandez", "maidenName": "", "age": 41, ...}
]
```

Now, suppose I want this data of console to render on the screen as well. Then we will use the 'map'.

App.jsx:

```
src > App.jsx > App > usersData.map() callback
1  import React, { useEffect, useState } from 'react'
2
3  function App() {
4    const [usersData, setUsersData] = useState([]);
5
6    useEffect(() => {
7      getUserData();
8    }, [])
9
10   async function getUserData() {
11     const url = "https://dummyjson.com/users";
12     let response = await fetch(url);
13     response = await response.json();
14     setUsersData(response.users);
15   }
16
17   console.log(usersData);
18
19   return (
20     <div>
21       <h1>Fetch data from API</h1>
22       {usersData.map((user) => (
23         <ul key={user.id}>
24           <li>{user.firstName}</li>
25           <li>{user.lastName}</li>
26         </ul>
27       ))}
28     </div>
29   )
30 }
31 export default App
```

Output:

Fetch data from API

- Emily
- Johnson
- Michael
- Williams
- Sophia
- Brown
- James
- Davis
- Emma
- Miller
- Olivia
- Wilson
- Alexander
- Jones

This React component uses `useState` to store user data in the `usersData` state variable, which starts as an empty array. When the component first loads, `useEffect` calls the `getUserData()` function, which fetches user information from the DummyJSON API, converts the response into JSON, and saves the users array into state using `setUsersData()`. Updating the state causes React to re-render the component, and the `map()` function loops through each user in `usersData`, displaying their first and last names inside a list. The `key={user.id}` helps React efficiently track and update each list item.

JSON server:

What is JSON?

JSON (JavaScript Object Notation) is a lightweight format used to store and exchange data between applications.

Example:

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@example.com"
}
```

Why JSON?

JSON is commonly used in APIs because it is easy for both humans and programs to read.

What is JSON Server?

JSON Server is a tool that creates a fake REST API from a JSON file. Instead of building a backend using Node.js, Express, Java, PHP, etc., you can quickly create APIs for frontend development and testing.

What is db.json?

db.json is the database file used by JSON Server.

How to install JSON server?

```
PS D:\ReactJS-revisited\learn-use-effect\React-useeffect-project> npm install json-server
added 42 packages, and audited 231 packages in 9s

56 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

How to Run JSON Server?

Open terminal in the project folder:

```
PS D:\ReactJS-revisited\learn-use-effect\React-useeffect-project> npx json-server db.json
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching db.json...

( ~ ^ _ ^ )

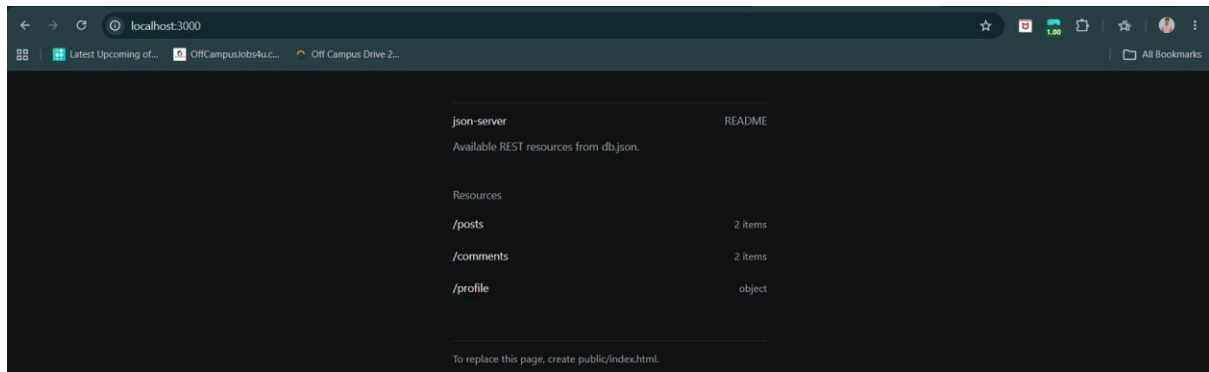
Index:
http://localhost:3000/

Static files:
Serving ./public directory if it exists

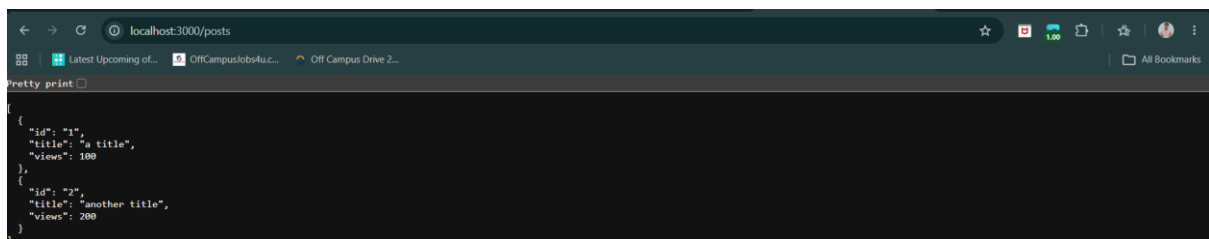
Endpoints:
http://localhost:3000/posts
http://localhost:3000/comments
http://localhost:3000/profile
```

Now, how to check if the APIs are there or not?

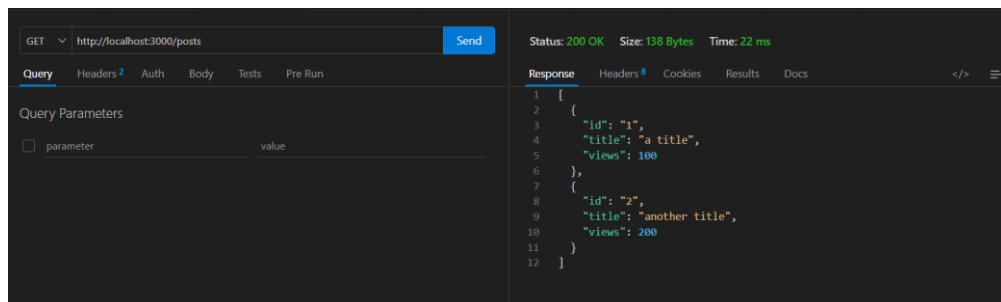
Copy-paste the local host link:



Click on any link:

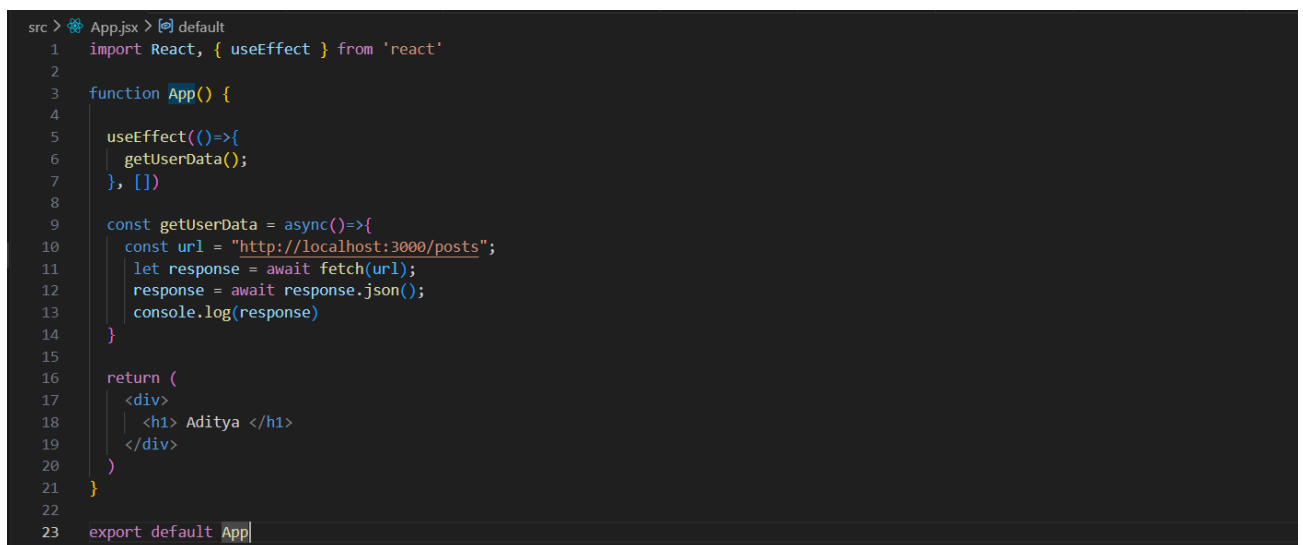


Yes, it worked. We can also test it in ThunderClient.

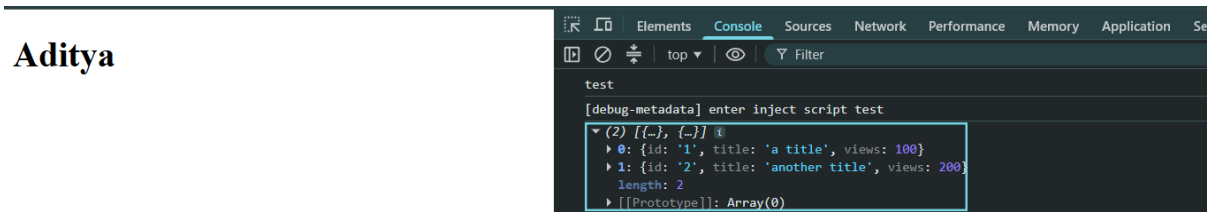


Now, how to make the API call? We can create the function for API call:

App.jsx: (we did the same in the API section)



Output:

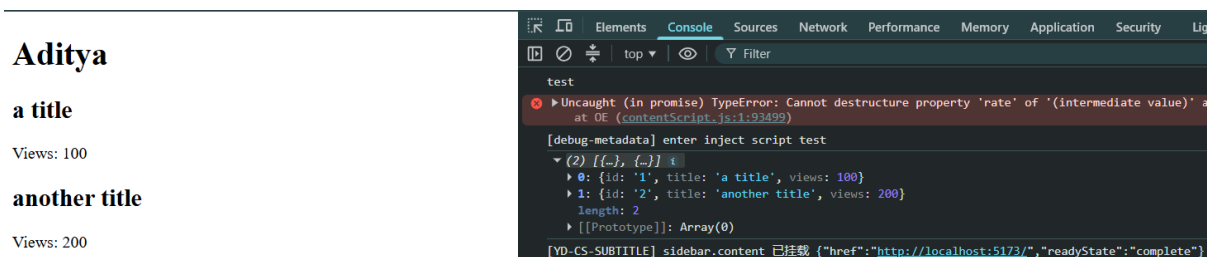


Now, we will render the content of the db.json on the screen, for this we will use the state and the map.

App.jsx:

```
src > App.jsx > App
1  import React, { useEffect, useState } from "react";
2
3  function App() {
4    const [userData, setUserData] = useState([]);
5
6    useEffect(() => {
7      getUserData();
8    }, []);
9
10   const getUserData = async () => {
11     const url = "http://localhost:3000/posts";
12     let response = await fetch(url);
13     response = await response.json();
14     setUserData(response);
15   };
16
17   return (
18     <div>
19       <h1>Aditya</h1>
20       {userData.map((user) => (
21         <div key={user.id}>
22           <h2>{user.title}</h2>
23           <p>Views: {user.views}</p>
24         </div>
25       ))}
26     </div>
27   );
28 }
29
30 export default App;
```

Output:



Now, adding the POST method in the code:

App.jsx:

```
src > App.jsx > App
1 import React, { useEffect, useState } from "react";
2
3 function App() {
4   const [userData, setUserData] = useState([]);
5   // States for the form
6   const [id, setId] = useState("");
7   const [title, setTitle] = useState("");
8   const [views, setViews] = useState("");
```

These three lines create **state variables** to store the values entered in the form's input fields. Initially, id, title, and views are empty strings (""). As the user types into each input, the corresponding setter function (setId, setTitle, or setViews) updates its value. This makes the inputs **controlled components**, meaning React always knows what the current value of each input is.

```
10   useEffect(() => {
11     getUserData();
12   }, []);
13   // GET Request
14   const getUserData = async () => {
15     const url = "http://localhost:3000/posts";
16     let response = await fetch(url);
17     response = await response.json();
18     setUserData(response);
19   };
```

```
20   // POST Request
21   const addPost = async () => {
22     const url = "http://localhost:3000/posts";
23
24     const response = await fetch(url, {
25       method: "POST",
26       headers: {
27         "Content-Type": "application/json",
28       },
29       body: JSON.stringify({
30         id,
31         title,
32         views: Number(views),
33       }),
34     });
```

This addPost function sends the data entered by the user to the server using a POST request. First, it stores the API URL in the url variable. Then, fetch() sends a request with method: "POST", telling the server to create a new post. The Content-Type: "application/json" header lets the server know that the data is being sent in JSON format. Finally, JSON.stringify() converts the JavaScript object containing id, title, and views into a JSON string before sending it, and Number(views) ensures that views is stored as a number instead of a string.

```
36   if (response.ok) {
37     // Clear the inputs
38     setId("");
39     setTitle("");
40     setViews("");
41     // Refresh the posts list
42     getUserData();
43     alert("Post Added Successfully!");
44   } else {
45     alert("Failed to add post.");
46   }
47 };
```

This code checks whether the POST request was successful using response.ok. If it is successful, it clears all three input fields by setting their state back to empty strings, calls getUserData() to fetch the updated list of posts (so the new post appears immediately), and shows a success message. If the request fails, it displays an alert informing the user that the post could not be added.

```
49   return (
50     <div style={{ padding: "20px" }}>
51       <h1>Aditya</h1>
52
53       /* Link to Add Post Section */
54       <a href="#add-post">+ Add New Post</a>
55
56       <hr />
57
58       /* Render Existing Posts */
59       {userData.map((user) => (
60         <div key={user.id}>
61           <h2>{user.title}</h2>
62           <p>Views: {user.views}</p>
63         </div>
64       ))}
65     </div>
66   );
```

```

68     /* Add Post Section */
69     <div id="add-post">
70       <h2>Add New Post</h2>
71       <input type="text" placeholder="Enter ID" value={id} onChange={(e) => setId(e.target.value)}
72       />
73       <br /><br />
74       <input type="text" placeholder="Enter Title" value={title} onChange={(e) => setTitle(e.target.value)}
75       />
76       <br /><br />
77       <input type="number" placeholder="Enter Views" value={views} onChange={(e) => setViews(e.target.value)}
78       />
79       <br /><br />
80       <button onClick={addPost}>Add Post</button>
81     </div>
82   </div>
83 </div>
84 </div>
85 </div>
86 </div>
87 export default App;

```

This section creates the Add New Post form. Each input field is a controlled component, meaning its value is linked to a React state variable (id, title, and views). As the user types, the onChange event updates the corresponding state using setId, setTitle, or setViews, ensuring React always has the latest input values. Finally, when the Add Post button is clicked, it calls the addPost() function, which sends the entered data to the server and adds the new post.

Now, the DELETE method:

App.jsx:

```

src > App.jsx > App
1  import React, { useEffect, useState } from "react";
2
3  function App() {
4    const [userData, setUserData] = useState([]);
5    const [id, setId] = useState("");
6
7    useEffect(() => {
8      getUserData();
9    }, []);
10   // GET Request
11   const getUserData = async () => {
12     const url = "http://localhost:3000/posts";
13     let response = await fetch(url);
14     response = await response.json();
15     setUserData(response);
16   };
17   // DELETE Request
18   const deletePost = async () => {
19     const url = `http://localhost:3000/posts/${id}`;
20     const response = await fetch(url, {
21       method: "DELETE",
22     });
23     if (response.ok) {
24       setId("");
25       getUserData();
26       alert("Post Deleted Successfully!");
27     } else {
28       alert("Failed to delete post.");
29     }
30   };

```

This deletePost function sends a **DELETE request** to remove a specific post from the server. It creates the URL by appending the entered id to the /posts endpoint, so only that post is targeted (for example, /posts/2). The fetch() function sends the request with method: "DELETE". After the request completes, response.ok checks whether the deletion was successful. If it is, the ID input field is cleared, getUserData() is called to refresh the list of posts so the deleted post disappears from the UI, and a success message is displayed. If the request fails, an error message is shown to inform the user.


```

32   return (
33     <div style={{ padding: "20px" }}>
34       <h1>Aditya</h1>
35       <a href="#delete-post"> Delete Post</a>
36       <hr />
37       {/* Render Existing Posts */}
38       {userData.map((user) => (
39         <div key={user.id}>
40           <h2>{user.title}</h2>
41           <p>Views: {user.views}</p>
42         </div>
43       ))}
44       <hr />
45       {/* Delete Post Section */}
46       <div id="delete-post">
47         <h2>Delete Post</h2>
48         <input
49           type="text"
50           placeholder="Enter Post ID"
51           value={id}
52           onChange={(e) => setId(e.target.value)}
53         />
54         <br />
55         <br />
56         <button onClick={deletePost}>Delete Post</button>
57       </div>
58     </div>
59   );
60 }
61
62 export default App;

```

Now, adding the PUT method to edit the posts the snippet:

```

21 // PUT Request
22 const editPost = async () => {
23   const url = `http://localhost:3000/posts/${id}`;
24
25   const response = await fetch(url, {
26     method: "PUT",
27     headers: {
28       "Content-Type": "application/json",
29     },
30     body: JSON.stringify({
31       id,
32       title,
33       views: Number(views),
34     }),
35   });
36
37   if (response.ok) {
38     setId("");
39     setTitle("");
40     setViews("");
41
42     getUserData();
43
44     alert("Post Updated Successfully!");
45   } else {
46     alert("Failed to update post.");
47   }
48 };

```

Now, adding the PATCH:

```
20
21 // PATCH Request
22 const editPost = async () => {
23   const url = `http://localhost:3000/posts/${id}`;
24
25   const response = await fetch(url, {
26     method: "PATCH",
27     headers: {
28       "Content-Type": "application/json",
29     },
30     body: JSON.stringify({
31       title,
32       views: Number(views),
33     }),
34   });
35
36   if (response.ok) {
37     setId("");
38     setTitle("");
39     setViews("");
40
41     getUserData();
42
43     alert("Post Updated Successfully!");
44   } else {
45     alert("Failed to update post.");
46   }
47 };
```